

Lab 6- Creating Tests for Inventory and Student projects.

Note: You should add the necessary dependencies for the tests as needed.

1. Open your inventory project.
2. In your test folder, create a new test class.
3. Add the following codes. Run it and make sure all tests are running. The following is an example of a unit test. We are testing the controller.

```
package com.example.inventory; // change this with your package

import com.example.inventory.controller.CategoryController;
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

import com.example.inventory.model.Category;
import com.example.inventory.repository.CategoryRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;
import org.springframework.ui.Model;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

import java.util.Collections;
import java.util.List;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

class CategoryControllerTest {

    @Mock
    private CategoryRepository categoryRepo;

    @Mock
    private Model model;

    @Mock
    private RedirectAttributes redirectAttributes;

    @InjectMocks
    private CategoryController controller;

    @BeforeEach
    void setUp() {
        MockitoAnnotations.openMocks(this);
    }

    @Test
    void testListShouldAddCategoryListToModel() {
        List<Category> categories = List.of(new Category());
```

```

        when(categoryRepo.findAllOrdered()).thenReturn(categories);

        String view = controller.list(model);

        verify(model).addAttribute("categorylist", categories);
        assertEquals("category", view);
    }

    // Test: add() - when category already exists

    @Test
    void testAddWhenCategoryAlreadyExists() {
        when(categoryRepo.existsById("A001")).thenReturn(true);

        String view = controller.add("A001", "Books",
redirectAttributes);

        verify(redirectAttributes).addFlashAttribute("errorMessage",
"Category code already exists.");
        assertEquals("redirect:/category", view);
        verify(categoryRepo, never()).save(any());
    }

    // Test: add() - successful adding

    @Test
    void testAddWhenCategoryIsNew() {
        when(categoryRepo.existsById("A002")).thenReturn(false);

        String view = controller.add("A002", "Electronics",
redirectAttributes);

        verify(categoryRepo).save(any(Category.class));
        verify(redirectAttributes).addFlashAttribute("successMessage",
"Category added.");
        assertEquals("redirect:/category", view);
    }

    // Test: showEdit() - category found

    @Test
    void testShowEditWhenCategoryFound() {
        Category c = new Category();
        c.setCatcode("A003");
        c.setCatdesc("Toys");

        when(categoryRepo.findById("A003")).thenReturn(Optional.of(c));

        String view = controller.showEdit("A003", model);

        verify(model).addAttribute("id", "A003");
        verify(model).addAttribute("desc", "Toys");
        assertEquals("catser", view);
    }

    // -----

```

```

-
    // Test: showEdit() - category not found

    @Test
    void testShowEditWhenCategoryNotFound() {
        when(categoryRepo.findById("X001")).thenReturn(Optional.empty());

        String view = controller.showEdit("X001", model);

        verify(model).addAttribute("errorMessage", "Category not
found.");
        assertEquals("redirect:/category", view);
    }

    // Test: edit() - duplicate new code

    @Test
    void testEditWithDuplicateNewCode() {
        when(categoryRepo.existsById("NEW")).thenReturn(true);

        String view = controller.edit("OLD", "NEW", "NewDesc",
redirectAttributes);

        verify(redirectAttributes).addFlashAttribute("errorMessage",
"New category code already exists.");
        assertEquals("redirect:/category", view);
    }

    // Test: edit() - success

    @Test
    void testEditSuccess() {
        when(categoryRepo.existsById("NEW")).thenReturn(false);
        when(categoryRepo.updateCategory("OLD", "NEW",
"NewDesc")).thenReturn(1);

        String view = controller.edit("OLD", "NEW", "NewDesc",
redirectAttributes);

        verify(redirectAttributes).addFlashAttribute("successMessage",
"Category updated.");
        assertEquals("redirect:/category", view);
    }

    // Test: delete() - has child items

    @Test
    void testDeleteWhenHasChildren() {
        when(categoryRepo.countItems("A001")).thenReturn(3);

        String view = controller.delete("A001", redirectAttributes);

        verify(redirectAttributes).addFlashAttribute("errorMessage",

```

```

"Cannot delete: category has 3 item(s). Delete items first.");
    verify(categoryRepo, never()).deleteById(anyString());
    assertEquals("redirect:/category", view);
}

// Test: delete() - success
@Test
void testDeleteSuccess() {
    when(categoryRepo.countItems("A002")).thenReturn(0);

    String view = controller.delete("A002", redirectAttributes);

    verify(categoryRepo).deleteById("A002");
    verify(redirectAttributes).addFlashAttribute("successMessage",
"Category deleted.");
    assertEquals("redirect:/category", view);
}
// Test: items() - category not found

@Test
void testItemsWhenCategoryNotFound() {

    when(categoryRepo.findWithItems("X001")).thenReturn(Optional.empty());

    String view = controller.items("X001", model);

    verify(model).addAttribute("errorMessage", "Category not
found.");
    assertEquals("redirect:/category", view);
}

// Test: items() - category has no items / Some extra tests

@Test
void testItemsWhenNoItems() {
    Category c = new Category();
    c.setCatcode("A001");
    c.setCatdesc("Books");
    c.setItems(Collections.emptyList());

    when(categoryRepo.findWithItems("A001")).thenReturn(Optional.of(c));

    String view = controller.items("A001", model);

    verify(model).addAttribute("errorMessage", "No items for this
category.");
    assertEquals("redirect:/category", view);
}

// Test: items() - success

@Test
void testItemsSuccess() {
    Category c = new Category();
    c.setCatcode("A001");

```

```

        c.setCatdesc("Books");
        c.setItems(List.of(new com.example.inventory.model.Item()));

        when(categoryRepo.findWithItems("A001")).thenReturn(Optional.of(c));

        String view = controller.items("A001", model);

        verify(model).addAttribute("desc", "Books");
        verify(model).addAttribute("itemlist", c.getItems());
        assertEquals("items", view);
    }
}

```

4. Create another test class and add the following. This is an integration testing test. Integration tests ensure that the web routes, form parameters, and redirects are all wired correctly when the application runs inside Spring.

```

@SpringBootTest
@AutoConfigureMockMvc
class CategoryControllerWithMockitoTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private CategoryController controller;

    private CategoryRepository mockRepo;

    @BeforeEach
    void setup() {
        // Create a pure Mockito mock
        mockRepo = Mockito.mock(CategoryRepository.class);
        // Inject mock into the controller (replaces the real
        repository)
        ReflectionTestUtils.setField(controller, "categoryRepo",
        mockRepo);
    }

    // testListDisplaysCategories

    @Test
    void testListDisplaysCategories() throws Exception {
        List<Category> fakeList = List.of(new Category());
        when(mockRepo.findAllOrdered()).thenReturn(fakeList);

        mockMvc.perform(get("/category"))
            .andExpect(status().isOk())
            .andExpect(view().name("category"))
            .andExpect(model().attributeExists("categorylist"))
            .andExpect(model().attribute("categorylist",
        fakeList));
    }
}

```

```

        verify(mockRepo).findAllOrdered();
    }

    // testAddCategoryAddsData
    @Test
    void testAddCategoryAddsData() throws Exception {
        when(mockRepo.existsById("C002")).thenReturn(false);

        mockMvc.perform(post("/add-todo")
            .param("catcode", "C002")
            .param("catdesc", "Electronics"))
            .andExpect(status().is3xxRedirection())
            .andExpect(redirectedUrl("/category"));

        ArgumentCaptor<Category> captor =
        ArgumentCaptor.forClass(Category.class);
        verify(mockRepo).save(captor.capture());

        Category saved = captor.getValue();
        assertThat(saved.getCatcode()).isEqualTo("C002");
        assertThat(saved.getCatdesc()).isEqualTo("Electronics");
    }

    // testEditCategoryUpdatesData
    @Test
    void testEditCategoryUpdatesData() throws Exception {
        when(mockRepo.existsById("C002")).thenReturn(false);
        when(mockRepo.updateCategory("C001", "C002",
        "Updated")).thenReturn(1);

        mockMvc.perform(post("/update-todo")
            .param("oldId", "C001")
            .param("catcode", "C002")
            .param("catdesc", "Updated"))
            .andExpect(status().is3xxRedirection())
            .andExpect(redirectedUrl("/category"));

        verify(mockRepo).updateCategory("C001", "C002", "Updated");
    }

    // testDeleteCategoryRemovesData
    @Test
    void testDeleteCategoryRemovesData() throws Exception {
        when(mockRepo.countItems("C001")).thenReturn(0);

        mockMvc.perform(get("/delete-todo").param("id", "C001"))
            .andExpect(status().is3xxRedirection())
            .andExpect(redirectedUrl("/category"));

        verify(mockRepo).deleteById("C001");
    }
}

```

5. Open your student project. Create a test class and add the following tests. Below is a combined unit and integration test.

```
package com.example.student;

import com.example.student.model.Student;
import com.example.student.web.StudentController;
import com.example.student.web.StudentService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.ArgumentCaptor;
import org.mockito.Mockito;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;

import org.springframework.test.util.ReflectionTestUtils;
import org.springframework.test.web.servlet.MockMvc;

import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.List;

import static org.assertj.core.api.Assertions.assertThat;
import static org.mockito.ArgumentMatchers.*;
import static org.mockito.Mockito.*;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@SpringBootTest
@AutoConfigureMockMvc
class StudentControllerIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private StudentController controller;

    private StudentService mockService;

    private final SimpleDateFormat df = new SimpleDateFormat("yyyy-MM-dd");

    @BeforeEach
    void setup() {
```

```

        mockService = Mockito.mock(StudentService.class);
        ReflectionTestUtils.setField(controller, "service", mockService);
    }

    // -----
    // Redirect root ("/") to /students
    // -----
    @Test
    void testHomeRedirectsToStudents() throws Exception {
        mockMvc.perform(get("/"))
            .andExpect(status().is3xxRedirection())
            .andExpect(redirectedUrl("/students"));
    }

    // -----
    // List and search students
    // -----
    @Test
    void testListDisplaysStudents() throws Exception {
        List<Student> fakeStudents = List.of(
            new Student(1L, "Alice", df.parse("2000-01-01"), true, 3.8)
        );

        when(mockService.search(null)).thenReturn(fakeStudents);
        mockMvc.perform(get("/students"))
            .andExpect(status().isOk())
            .andExpect(model().attributeExists("students"))
            .andExpect(model().attributeExists("keyword"))
            .andExpect(model().attributeExists("start"))
            .andExpect(model().attributeExists("limit"))
            .andExpect(model().attributeExists("total"));

        verify(mockService).search(null);
    }

    // -----
    // Show create form
    // -----
    @Test
    void testShowCreateForm() throws Exception {
        mockMvc.perform(get("/students/new"))
            .andExpect(status().isOk())
            .andExpect(view().name("student-form"))
            .andExpect(model().attributeExists("student"));
    }

    // -----
    // Create student - success
    // -----
    @Test
    void testCreateStudentSuccess() throws Exception {

```

```

        mockMvc.perform(post("/students")
            .param("name", "John Doe")
            .param("dob", "2000-05-05")
            .param("gpa", "3.5")
            .param("passed", "true"))
            .andExpect(status().is3xxRedirection())
            .andExpect(redirectedUrl("/students"));

        ArgumentCaptor<Student> captor = ArgumentCaptor.forClass(Student.class);
        verify(mockService).save(captor.capture());

        Student saved = captor.getValue();
        assertThat(saved.getName()).isEqualTo("John Doe");
        assertThat(saved.getDob()).isEqualTo(df.parse("2000-05-05"));
        assertThat(saved.getGpa()).isEqualTo(3.5);
        assertThat(saved.isPassed()).isTrue();
    }

    // -----
    //  Create student - validation error (missing name)
    // -----
    @Test
    void testCreateStudentValidationErrorMissingName() throws Exception {
        mockMvc.perform(post("/students")
            .param("name", "") // invalid (NotBlank)
            .param("dob", "2000-05-05")
            .param("gpa", "3.2")
            .param("passed", "true"))
            .andExpect(status().isOk())
            .andExpect(view().name("student-form"));

        verify(mockService, never()).save(any());
    }

    // -----
    //  Create student - validation error (future date)
    // -----
    @Test
    void testCreateStudentValidationErrorFutureDOB() throws Exception {
        mockMvc.perform(post("/students")
            .param("name", "Future Kid")
            .param("dob", "2099-01-01") // invalid (must be past)
            .param("gpa", "3.0"))
            .andExpect(status().isOk())
            .andExpect(view().name("student-form"));

        verify(mockService, never()).save(any());
    }

    // -----
    //  Create student - validation error (GPA out of range)

```

```

// -----
@Test
void testCreateStudentValidationErrorGPAOutOfRange() throws Exception {
    mockMvc.perform(post("/students")
        .param("name", "Smartie")
        .param("dob", "2000-01-01")
        .param("gpa", "5.0")) // invalid (too high)
        .andExpect(status().isOk())
        .andExpect(view().name("student-form"));

    verify(mockService, never()).save(any());
}

// -----
//  Show edit form
// -----
@Test
void testShowEditForm() throws Exception {
    Student s = new Student(1L, "Jane", df.parse("1999-05-01"), false, 2.8);
    when(mockService.get(1L)).thenReturn(s);

    mockMvc.perform(get("/students/1/edit"))
        .andExpect(status().isOk())
        .andExpect(view().name("student-form"))
        .andExpect(model().attribute("student", s));

    verify(mockService).get(1L);
}

// -----
//  Update student - success
// -----
@Test
void testUpdateStudentSuccess() throws Exception {
    mockMvc.perform(post("/students/1")
        .param("name", "Jane Updated")
        .param("dob", "2000-01-01")
        .param("gpa", "3.9")
        .param("passed", "true"))
        .andExpect(status().is3xxRedirection())
        .andExpect(redirectedUrl("/students"));

    ArgumentCaptor<Student> captor = ArgumentCaptor.forClass(Student.class);
    verify(mockService).save(captor.capture());
    Student updated = captor.getValue();

    assertThat(updated.getId()).isEqualTo(1L);
    assertThat(updated.getName()).isEqualTo("Jane Updated");
    assertThat(updated.getGpa()).isEqualTo(3.9);
    assertThat(updated.isPassed()).isTrue();
}

```

```

// -----
//  Update student - validation error (empty name)
// -----
@Test
void testUpdateStudentValidationError() throws Exception {
    mockMvc.perform(post("/students/1")
        .param("name", "")
        .param("dob", "2000-01-01")
        .param("gpa", "3.0"))
        .andExpect(status().isOk())
        .andExpect(view().name("student-form"));

    verify(mockService, never()).save(any());
}

// -----
//  Delete student
// -----
@Test
void testDeleteStudent() throws Exception {
    mockMvc.perform(post("/students/1/delete"))
        .andExpect(status().is3xxRedirection())
        .andExpect(redirectedUrl("/students"));

    verify(mockService).delete(1L);
}

// -----
//  Search by keyword
// -----
@Test
void testSearchStudents() throws Exception {
    List<Student> fakeStudents = List.of(
        new Student(2L, "Mark", df.parse("2001-02-02"), true, 3.2)
    );

    when(mockService.search("Mark")).thenReturn(fakeStudents);

    mockMvc.perform(get("/students").param("keyword", "Mark"))
        .andExpect(status().isOk())
        .andExpect(view().name("students"))
        .andExpect(model().attributeExists("students"))
        .andExpect(model().attribute("keyword", "Mark"));

    verify(mockService).search("Mark");
}
}

```